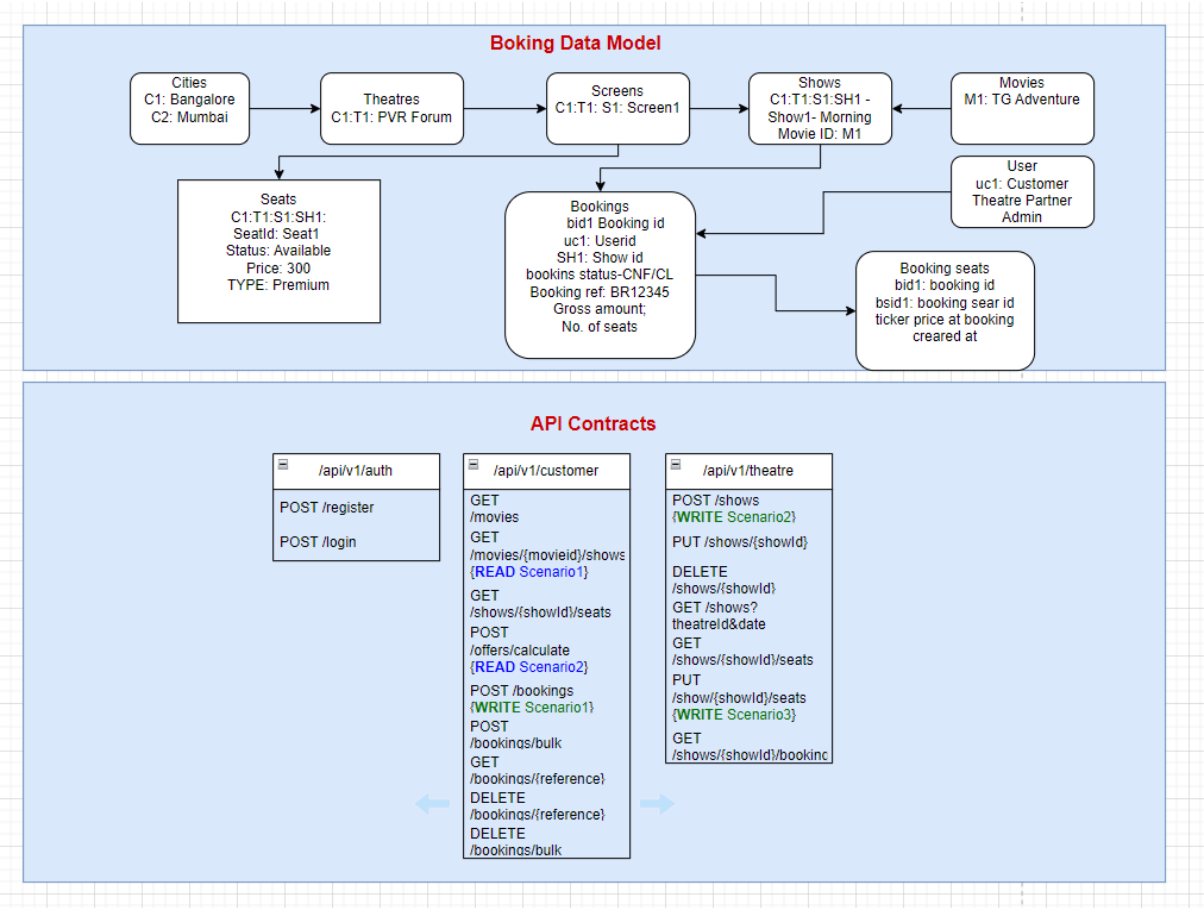
Boking Data Model
Cities
C1: Bangalore
C2: Mumbai
Theatres
C1:T1: PVR Forum
Screens
C1:T1: S1: Screen1
Shows
C1:T1:S1:SH1 -
Show1- Morning
Movie ID: M1
Movies
M1: TG Adventure
Seats
C1:T1:S1:SH1:
SeatId: Seat1
Status: Available
Price: 300
TYPE: Premium
Bookings
bid1 Booking id
uc1: Userid
SH1: Show id
bookins status-CNF/CL
Booking ref: BR12345
Gross amount;
No. of seats
User
uc1: Customer
Theatre Partner
Admin
Booking seats
bid1: booking id
bsid1: booking sear id
ticker price at booking
creared at

API Contracts
/api/v1/auth
POST /register
POST /login
/api/v1/customer
GET
/movies
GET
/movies/{movieid}/shows
{READ Scenario1}
GET
/shows/{showId}/seats
POST
/offers/calculate
{READ Scenario2}
POST /bookings
{WRITE Scenario1}
POST
/bookings/bulk
GET
/bookings/{reference}
DELETE
/bookings/{reference}
DELETE
/bookings/bulk
/api/v1/theatre
POST /shows
{WRITE Scenario2}
PUT /shows/{showId}
DELETE
/shows/{showId}
GET /shows?
theatreId&date
GET
/shows/{showId}/seats
PUT
/show/{showId}/seats
{WRITE Scenario3}
GET
/shows/{showId}/bookinc

Design patterns

Strategy Pattern — discount calculation `OfferService` holds a `List<DiscountStrategy>` injected by Spring. At booking time every strategy's `isApplicable()` is checked and all matching discounts are **stacked** — both the afternoon and third-ticket discounts can fire together on the same booking. Adding a new offer requires only a new `@Component` class; no existing code changes.

```
DiscountStrategy (interface)
  ├── AfternoonShowDiscountStrategy   — 20% off shows 12:00–16:00
  └── ThirdTicketDiscountStrategy     — 50% off every 3rd ticket (price-desc sort)
```

Repository Pattern All data access is behind `JpaRepository` interfaces with named JPQL queries. Business logic in services never builds queries inline.

DTO Pattern Separate request and response classes for every API surface. Entities never leave the service layer.

Key entity decisions:

- **Seats are screen-scoped**, not show-scoped. Seat records are created once when a theatre partner allocates inventory. `BookingSeat` is the join table that links a booking to specific seats for a specific show, with a `ticketPriceAtBooking` snapshot so price changes never affect issued receipts.
- **`@Version` on `Show` and `Booking`** provides optimistic locking against lost-update on the `availableSeatCount` field.
- **Pessimistic write lock** (`PESSIMISTIC_WRITE`) on the seat query inside `BookingService` prevents two concurrent transactions from booking the same seat simultaneously.
- **`Show.ShowStatus`** enum (`SCHEDULED → RUNNING → COMPLETED / CANCELLED`) tracks lifecycle alongside the `isActive` boolean used in query filters.
- **`Booking.PaymentStatus`** (`PENDING / COMPLETED / FAILED / REFUNDED`) is tracked independently from `BookingStatus` so a confirmed booking can still have a pending payment while awaiting a gateway callback.

```java
 * Core booking flow:
 * 1. Validate the show is still accepting bookings
 * 2. Acquire a pessimistic write lock on selected seats — prevents two
 *    concurrent transactions from booking the same seat
 * 3. Verify each locked seat is still available for this show
 * 4. Calculate gross total from per-seat prices (PREMIUM/VIP cost more)
 * 5. Stack all applicable discount strategies via OfferService
 * 6. Persist booking + BookingSeat records atomically
 * 7. Decrement show.availableSeatCount
 */
```

```java
@PostMapping("/bookings")
@SecurityRequirement(name = "bearerAuth")
@Operation(summary = "Write Scenario 1 — book tickets with automatic offer application")
public ResponseEntity<BookingResponse> bookTickets(
        @Valid @RequestBody BookTicketsRequest request,
        @AuthenticationPrincipal UserDetails principal) {

    Long customerUserId = userResolverService.resolveCurrentUserId(principal);
    return ResponseEntity.status(HttpStatus.CREATED)
            .body(bookingService.bookSelectedTickets(customerUserId, request));
}
```

```java
/**
 * Core booking flow:
 * 1. Validate the show is still accepting bookings
 * 2. Acquire a pessimistic write lock on selected seats — prevents two
 *    concurrent transactions from booking the same seat
 * 3. Verify each locked seat is still available for this show
 * 4. Calculate gross total from per-seat prices (PREMIUM/VIP cost more)
 * 5. Stack all applicable discount strategies via OfferService
 * 6. Persist booking + BookingSeat records atomically
 * 7. Decrement show.availableSeatCount
 */
@Transactional
public BookingResponse bookSelectedTickets(Long customerUserId, BookTicketsRequest request) {
        // Pessimistic lock — concurrent requests for the same seats queue here
    List<Seat> lockedSeats =
            seatRepository.findSeatsByIdWithPessimisticLock(request.getSelectedSeatIds());
```

```java
/**
 * Acquires a pessimistic write lock on the requested seats before any      You, 5
 * status check, preventing two concurrent transactions from booking
 * the same seat simultaneously.
 */
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT s FROM Seat s WHERE s.seatId IN :seatIds")
List<Seat> findSeatsByIdWithPessimisticLock(@Param("seatIds") List<Long> seatIds);
```

```java
        // Calculate pricing with all applicable discount strategies stacked
        // Build offer summary string for the booking record
        // Create seat association records with price snapshot
        // Reduce available count on the show
```

```java
    Booking booking = Booking.builder()
            .bookingReference(generateBookingReference())
            .customer(customer)
            .show(show)
            .numberOfSeats(lockedSeats.size())
            .grossAmount(pricing.getGrossAmount())
            .discountAmount(pricing.getTotalDiscountAmount())
            .netPayableAmount(pricing.getNetPayableAmount())
            .appliedOfferSummary(offerSummary.isEmpty() ? null : offerSummary)
            .bookingStatus(BookingStatus.CONFIRMED)
            .paymentStatus(Booking.PaymentStatus.PENDING)
            .bookingDate(LocalDateTime.now())
            .build();

    booking = bookingRepository.save(booking);
```